



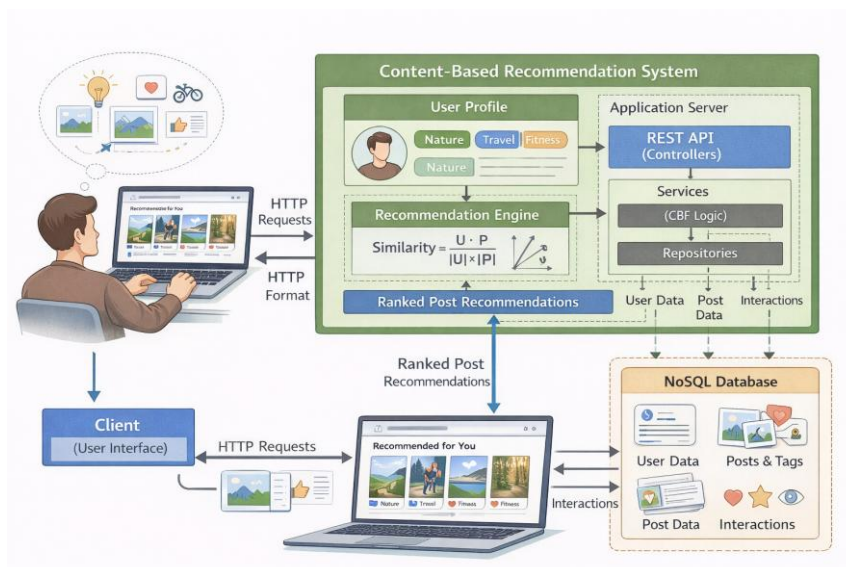
מכללת כנפי רוח - קרית נוער ירושלים

הצעת פרויקט לעבודת גמר

י"ד הנדסאי תובנה (שאלון: 714918)

בנושא

גלריית מיניאטורות gallery miniatures



שם הסטודנט: יהושע נאימרק

ת"ז: 329426464

מנחה: אילן פרץ

תאריך: 12/1/2026

מכללה: כנפי רוח, קרית נוער ירושלים (סמל מוסד: 140129)

סמל מוסד: 140129
 שם מכללה: כנפי רוח קריית נוער ירושלים
 שם הסטודנט: יהושע נאימרק
 ת"ז: 329426464
 שם הפרויקט: גלריית מיניאטורות gallery miniatures

תוכן עניינים

1. תיאור הנושא 3
2. רקע תיאורטי בתחום הפרויקט 3
3. תיאור הפרויקט 5
4. הגדרת הבעיה האלגוריתמית 6
6. הילכים עיקריים בתחום למידת מכונה (כולל ניתוח השוואתי) לא רלוונטי לפרויקט 11
7. הילכים עיקריים בתחום מערכות הפעלה\רשתות\אבטחה לא רלוונטי לפרויקט 11
8. פרוטוקולי תקשורת 11
9. פיתוחים עתידיים 13
10. תיאור טכנולוגיית הנדסה 14
11. מסד נתונים 15
12. פרטים פורמליים 18

1. תיאור הנושא

תחום שיתוף התוכן החזותי ברשת התפתח משמעותית בעידן הדיגיטלי, והוא כולל פלטפורמות המאפשרות העלאה, צפייה וארגון של תמונות, סרטונים ויצירות גרפיות. תחום זה משלב בין עולמות התוכן האומנותי לבין מערכות מידע, ניהול נתונים וחווית משתמש, ומהווה חלק מרכזי במערכות מדיה חברתית מודרניות.

אחד האתגרים המרכזיים בתחום הוא **ניהול וסינון של כמויות גדולות של תוכן חזותי**, תוך התאמה להעדפות שונות של משתמשים. משתמשים נחשפים למגוון רחב של יצירות, סגנונות וקטגוריות, ולעיתים מתקשים למצוא תוכן ממוקד ורלוונטי עבורם. מנגד, יוצרים מתקשים להגיע לקהל היעד המתאים לעבודותיהם בתוך עומס המידע הקיים.

מבחינה טכנולוגית, תחום זה עושה שימוש במערכות מבוססות רשת, מסדי נתונים מתקדמים ואלגוריתמים לניתוח מידע. המידע המנוהל במערכות אלו כולל טקסטים, תגיות, קטגוריות, קבצי מדיה ונתוני אינטראקציה של משתמשים. שילוב זה מחייב פתרונות הנדסיים יעילים לאחסון, שליפה, עיבוד והצגת מידע.

בנוסף, קיימת חשיבות רבה ליכולת של המערכת להבין מאפיינים של תוכן חזותי ושל תחומי עניין כלליים, גם ללא שימוש במודלים מורכבים של למידת מכונה. פתרונות אלגוריתמיים קלאסיים מאפשרים ניתוח מבני של נתונים, סיווג תוכן והצגת מידע בצורה ממוקדת ושקופה.

התחום מאופיין בדינמיות גבוהה: כמות התוכן גדלה באופן מתמיד, העדפות המשתמשים משתנות, וסוגי המדיה מתפתחים. לכן נדרש תכנון הנדסי שמאפשר גמישות, הרחבה עתידית ויכולת התאמה לשינויים, תוך שמירה על ביצועים ויעילות.

2. רקע תיאורטי בתחום הפרויקט

תחום שיתוף התוכן החזותי ברשת מהווה חלק מרכזי במערכות מידע מודרניות ובפלטפורמות מדיה חברתית. מערכות אלו נועדו לאפשר למשתמשים להעלות, לצפות ולצרוך תוכן ויזואלי כגון תמונות, סרטונים ויצירות גרפיות, תוך ניהול כמויות גדולות של מידע ואינטראקציות משתמשים.

ככל שכמות התוכן במערכות אלו גדלה, עולה הצורך במנגנונים חכמים לסינון, דירוג והתאמה אישית של מידע. ללא מנגנונים כאלה, המשתמש נחשף לעומס מידע המקשה על מציאת תוכן רלוונטי, ואילו יוצרים מתקשים להגיע לקהל היעד המתאים.

פתרונות קיימים בתחום

כיום קיימות פלטפורמות רבות לשיתוף תוכן חזותי, ביניהן רשתות חברתיות כלליות ופלטפורמות ייעודיות לאמנות ועיצוב. מערכות אלו משלבות מנגנוני המלצה מתקדמים, המבוססים לרוב על ניתוח התנהגות משתמשים בהיקף רחב.

רוב הפתרונות הקיימים נשענים על אחד או יותר מהאלגוריתמים הבאים:

1. אלגוריתמים מבוססי שיתוף פעולה (Collaborative Filtering)

אלגוריתמים אלו מנתחים דפוסי שימוש של משתמשים רבים, ומניחים שמשתמשים בעלי התנהגות דומה יאהבו תכנים דומים. גישה זו נפוצה בפלטפורמות גדולות, אך דורשת כמות גדולה של נתונים ואינטראקציות.

2. אלגוריתמים מבוססי תוכן (Content-Based Filtering)

גישה זו מתמקדת בניתוח מאפייני התוכן עצמו, כגון תגיות, טקסט, קטגוריות או מאפיינים חזותיים. ההמלצות מבוססות על התאמה בין מאפייני התוכן לבין תחומי עניין כלליים של המשתמש, ללא תלות במשתמשים אחרים.

3. מערכות היברידיות

מערכות אלו משלבות בין מספר שיטות, למשל שילוב בין ניתוח תוכן לבין ניתוח התנהגות משתמשים. פתרונות כאלה מספקים רמת דיוק גבוהה, אך מאופיינים במורכבות חישובית ותפעולית גבוהה.

מקורות תיאורטיים ואקדמיים (רקע כללי)

ספרות אקדמית בתחום מערכות ההמלצה מצביעה על כך שאין פתרון אחד שמתאים לכל מערכת. הבחירה באלגוריתם תלויה בגודל המערכת, בכמות המשתמשים, בסוג התוכן ובדרישות ההנדסיות כגון שקיפות, יציבות ויכולת הסבר של תהליך קבלת ההחלטות. מחקרים רבים מדגישים כי במערכות קטנות או ייעודיות, אלגוריתמים פשוטים ושקופים עשויים להיות יעילים יותר ממודלים מורכבים.

טבלת השוואה בין פתרונות קיימים

חסרונות	יתרונות	שם הפתרון / הגישה
עומס מידע, חוסר מיקוד בתחום ספציפי, חוסר שקיפות	חשיפה רחבה, אלגוריתמים מתקדמים, התאמה אישית גבוהה	רשתות חברתיות כלליות (כגון Instagram)
פחות התאמה אישית למשתמש הבודד, תלות בפופולריות	מיקוד בתוכן אומנותי, קהל רלוונטי	פלטפורמות אמנות ייעודיות (כגון ArtStation)
דורש הרבה משתמשים, בעיית Cold Start	לומד מדפוסי שימוש אמיתיים, מגלה קשרים חבויים	Collaborative Filtering
תלוי באיכות ניתוח התוכן, עלול לצמצם גיוון	אינו תלוי במשתמשים אחרים, שקוף ויציב	Content-Based Filtering
מורכבות גבוהה, משאבים רבים	דיוק גבוה, גמישות	מערכות היברידיות

ניתוח הפער הקיים בתחום

מהסקירה עולה כי הפתרונות הקיימים מתאימים בעיקר למערכות רחבות היקף עם בסיס משתמשים גדול ונתונים רבים. במערכות קטנות, ייעודיות או בתחילת דרכן, קיימת מגבלה ביכולת ליישם פתרונות מורכבים ללא תשתית נתונים מספקת.

בנוסף, מרבית המערכות הקיימות פועלות כ"קופסה שחורה", שבה קשה להסביר מדוע תוכן מסוים הוצג למשתמש. מצב זה בעייתי במיוחד בפרויקטים הנדסיים או לימודיים, שבהם נדרשת הבנה מלאה של תהליך קבלת ההחלטות.

לכן קיים פער בין פתרונות מסחריים גדולים לבין הצורך בפתרונות ממוקדים, שקופים וניתנים לניתוח, המיועדים לתחום תוכן מוגדר ולמערכות בקנה מידה קטן יותר. פער זה מצדיק פיתוח ויישום של פתרונות נוספים, המותאמים לצרכים אלו.

3. תיאור הפרויקט

פסקת פתיחה:

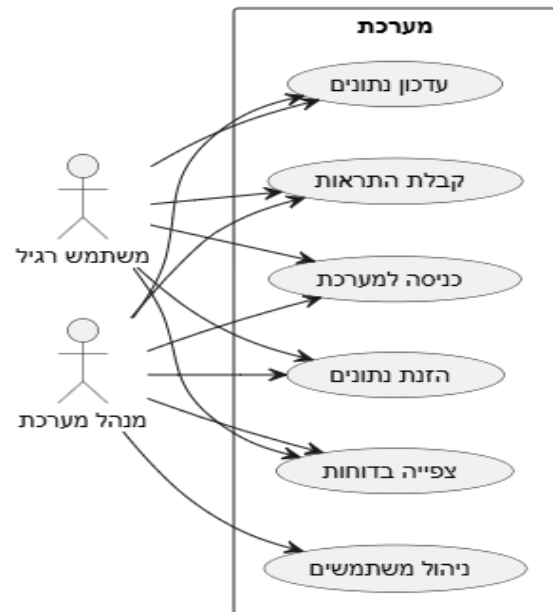
המערכת היא פלטפורמה שמיועדת ל[סוג המשתמשים – לדוגמה: תלמידים ומורים / לקוחות / מנהלי מערכת]. היא מאפשרת למשתמשים לבצע [משימות עיקריות – לדוגמה: לנהל הזמנות, להוסיף המלצות, לעקוב אחרי נתונים וכו'] בצורה קלה ומהירה דרך ממשק ידידותי. המשתמשים ניגשים למערכת דרך [דפדפן/אפליקציה], מבצעים את הפעולות שהם צריכים, והמערכת מטפלת ב[ניהול נתונים/חישובים/קבלת החלטות] מאחוריה באופן אוטומטי.

תיאור הרכיבים העיקריים מנקודת מבט המשתמש:

- **מסך הבית / לוח בקרה:** המשתמש רואה את כל הפעולות שהוא יכול לבצע בלחיצה אחת, עם מידע מרכזי ברור.
- **טפסים וכניסת מידע:** מאפשרים להכניס נתונים חדשים או לעדכן נתונים קיימים בקלות.
- **דוחות ותצוגות מידע:** המערכת מספקת סיכומים, גרפים ודוחות שמראים למשתמש את המידע החשוב ביותר.
- **התראות/הודעות:** אם קורה משהו שדורש תשומת לב, המערכת שולחת הודעה או מספקת התראה במסך.

איך המערכת פותרת את הבעיה:

המערכת מסייעת ב[תיאור הבעיה/אתגר מסעיף 1] על ידי [הסבר איך המערכת פותרת את הבעיה – לדוגמה: חוסכת זמן, מאגדת מידע במקום אחד, מונעת טעויות אנוש, מאפשרת ניהול חכם של המשימות וכו']. המשתמש יכול לבצע את הפעולות במהירות ובפשטות, בלי צורך בידע טכני מיוחד.



תרחיש לדוגמה – “הזנת נתונים”:

1. המשתמש נכנס למערכת.
2. בוחר באפשרות “הזנת נתונים”.
3. ממלא את הטופס עם המידע הנדרש.
4. לוחץ על “שמור”.
5. המערכת מאמתת את הנתונים ושומרת אותם בבסיס הנתונים.
6. המשתמש מקבל הודעת אישור שהמידע נוסף בהצלחה.

4. הגדרת הבעיה האלגוריתמית

בסעיף **תיאור הנושא** תואר האתגר של עומס תוכן חזותי במערכות שיתוף, אשר מקשה על משתמשים לאתר פוסטים התואמים את תחומי העניין שלהם. ריבוי הפוסטים, הגיוון הסגנוני והיעדר הגדרה מפורשת של העדפות המשתמש יוצרים קושי בהצגת תוכן רלוונטי וממוקד.

מתוך אתגר זה נגזרת הבעיה האלגוריתמית של הפרויקט:

הבעיה האלגוריתמית:

פיתוח אלגוריתם מבוסס Content-Based Filtering שמטרתו לחשב ולדרג את מידת ההתאמה בין משתמש לבין פוסטים במערכת, על סמך מאפייני התוכן בלבד, כך שיוצגו למשתמש הפוסטים הרלוונטיים ביותר לתחומי העניין שלו.

הבעיה מוגדרת כבעיה אלגוריתמית מסוג **התאמה ודירוג תוכן**, שבה כל פוסט מיוצג באמצעות מאפיינים קבועים (כגון קטגוריות, תגיות ומילות מפתח), והעדפות המשתמש מיוצגות באמצעות

ניתוח אינטראקציות קודמות עם פוסטים. האלגוריתם נדרש להשוות בין ייצוגים אלו בצורה פורמלית ולחשב ערך התאמה המאפשר מיון והצגת פוסטים בסדר יורד של רלוונטיות.

קשר לאתגרי התחום

בעיה אלגוריתמית זו נובעת ישירות מהאתגרים שתוארו בסעיף **תיאור הנושא**: הצורך להתמודד עם עומס מידע חזותי, לאפשר התאמה אישית במערכת בעלת היקף משתמשים מוגבל, ולספק פתרון הנדסי שקוף ובר-הסבר. בחירה באלגוריתם Content-Based Filtering מאפשרת מענה לאתגרים אלו באמצעות חישובים מתמטיים והשוואת מאפייני תוכן, ללא תלות בנתוני משתמשים אחרים.

האלגוריתם המרכזי במערכת (CBF) – Content-Based Filtering

מטרת האלגוריתם

מטרת אלגוריתם הליבה במערכת היא **להתאים תוכן חזותי רלוונטי לכל משתמש**, מתוך מאגר גדול של פוסטים, על סמך תחומי העניין האישיים שלו. האלגוריתם נועד לצמצם עומס מידע ולהציג למשתמש פוסטים הדומים לתכנים שבהם גילה עניין בעבר.

בעיה זו נובעת ישירות מהאתגרים שתוארו בסעיף **תיאור הנושא**: ריבוי תוכן חזותי, קושי במיקוד, והיעדר מידע מוקדם מספק על העדפות המשתמש.

סוג האלגוריתם

האלגוריתם שנבחר הוא – **Content-Based Filtering (CBF)** האלגוריתם קלאסי ממערכות המלצה, שאינו מבוסס למידת מכונה.

האלגוריתם פועל על העיקרון:

משתמש נוטה להתעניין בתוכן הדומה לתוכן שבו התעניין בעבר.

אופן פעולת האלגוריתם (שלבים לוגיים)

שלב 1 – ייצוג פוסטים כוקטורי תכונות

כל פוסט במערכת מנותח ומיוצג כוקטור תכונות מספרי.

דוגמאות לתכונות:

- קטגוריות
- תגיות
- מילות מפתח בתיאור
- מאפיינים חזותיים כלליים

כל תכונה מיוצגת כערך מספרי (0/1 או משקל), וכך מתקבל:

$$Post = (f_1, f_2, f_3, \dots, f_n)$$

שלב 2 – בניית פרופיל משתמש

פרופיל המשתמש נבנה על סמך האינטראקציות שלו עם פוסטים:

- צפיות
- לייקים
- סימון מועדפים

פרופיל המשתמש מחושב כוקטור ממוצע של וקטורי הפוסטים שאיתם המשתמש ביצע אינטראקציה:

$$User = \frac{1}{k} \sum_{i=1}^k Post_i$$

כאשר k הוא מספר הפוסטים הרלוונטיים למשתמש.

הפרופיל דינמי ומתעדכן בכל פעולה חדשה של המשתמש.

שלב 3 – חישוב דמיון מתמטי

לצורך מדידת מידת ההתאמה בין משתמש לפוסט, נעשה שימוש במדד **Cosine Similarity**.
הנוסחה:

$$Similarity(U, P) = \frac{U \cdot P}{\|U\| \cdot \|P\|}$$

פירוש:

- ערך קרוב ל-1 → התאמה גבוהה
- ערך קרוב ל-0 → התאמה נמוכה

שלב 4 – דירוג והמלצה

- האלגוריתם מחשב ציון התאמה לכל פוסט
- הפוסטים ממוינים לפי הציון
- מוצגים למשתמש רק הפוסטים בעלי ההתאמה הגבוהה ביותר

למה נבחר אלגוריתם CBF?

הבחירה ב־Content-Based Filtering נעשתה משיקולים הנדסיים ברורים:

יתרונות:

- ✓ אינו תלוי במספר גדול של משתמשים

- ✓פתרון שקוף וניתן להסבר
- ✓מתאים למערכות קטנות או ייעודיות
- ✓אינו דורש למידת מכונה או אימון מודלים
- ✓קל לבדיקה, בקרה ותחזוקה

התאמה לפרויקט:

- המערכת נמצאת בשלב מוקדם
- כמות המשתמשים מוגבלת
- נדרש פתרון דטרמיניסטי ובר-הסבר
- מתאים לדרישות פרויקט גמר בהנדסת תוכנה

סיכום האלגוריתם

אלגוריתם הליבה בפרויקט מבוסס על:

- ייצוג תוכן באמצעות תכונות
- בניית פרופיל משתמש דינמי
- חישובי דמיון מתמטיים
- דירוג תכנים מותאם אישית

הפתרון מספק מערכת המלצות יעילה, ברורה ויציבה, המותאמת להיקף ולמטרות הפרויקט.

5. הליכים עיקריים בפתרון הבעיה בטכנולוגיות הנדסה מתקדמות

סעיף זה מתאר את מבנה המערכת ואת אופן פעולתה מנקודת מבט הנדסית-טכנית, כלומר מנקודת מבטו של המפתח. בניגוד לסעיף **תיאור הפרויקט**, המתמקד בפעולות המשתמש, סעיף זה עוסק בארכיטקטורה של המערכת, בזרימת המידע ובאופן שבו האלגוריתם מופעל בפועל.

המערכת בנויה כמערכת מבוססת רשת, הפועלת על פי מודל של שלוש שכבות: שכבת מצגת, שכבת לוגיקה עסקית ושכבת נתונים. מבנה זה מאפשר הפרדה ברורה בין רכיבי המערכת, תחזוקה קלה והרחבה עתידית.

ארכיטקטורת המערכת וזרימת המידע

המערכת כוללת את הרכיבים ההנדסיים הבאים:

- לקוח (Client / Front-End)
אחראי להצגת המידע למשתמש ולקבלת פעולות כגון צפייה בפוסטים, לייקים וסימון מועדפים.
- שרת יישומים (Back-End / API Server)
אחראי על עיבוד הבקשות, ניהול הלוגיקה העסקית והפעלת אלגוריתם-Content Based Filtering.
- מוד נתונים (Database – NoSQL)
אחראי על אחסון נתוני משתמשים, פוסטים והיסטוריית אינטראקציות.
זרימת מידע כללית:

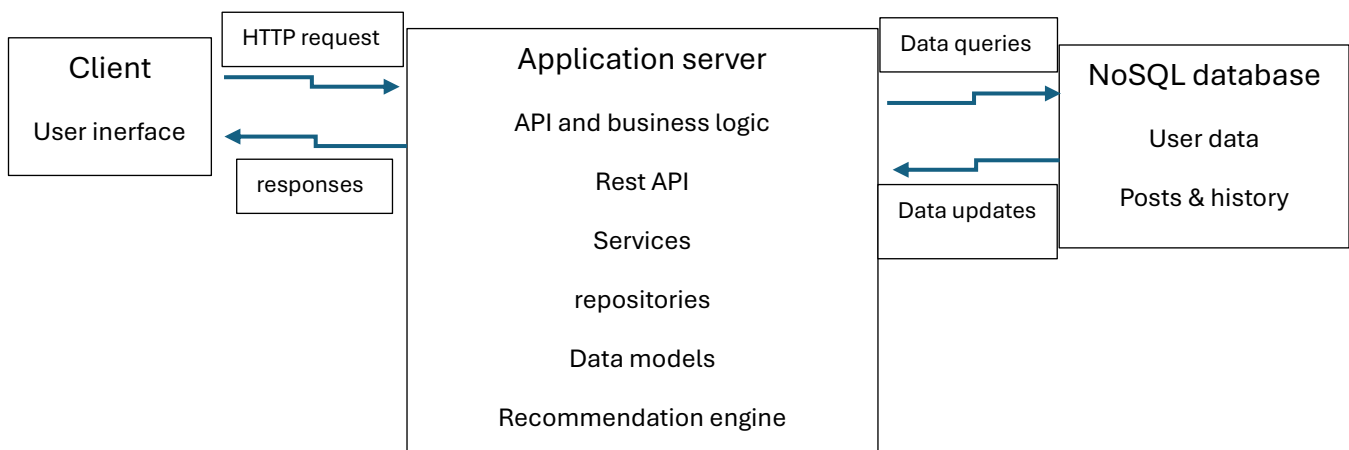
1. המשתמש מבצע פעולה בממשק (לדוגמה: כניסה למסך הבית).

2. שכבת המצגת שולחת בקשת HTTP לשרת.

3. השרת שולף נתונים רלוונטיים ממסד הנתונים.

4. מופעל מנגנון ההמלצות לצורך דירוג פוסטים.

5. התוצאות מוחזרות ללקוח ומוצגות למשתמש.



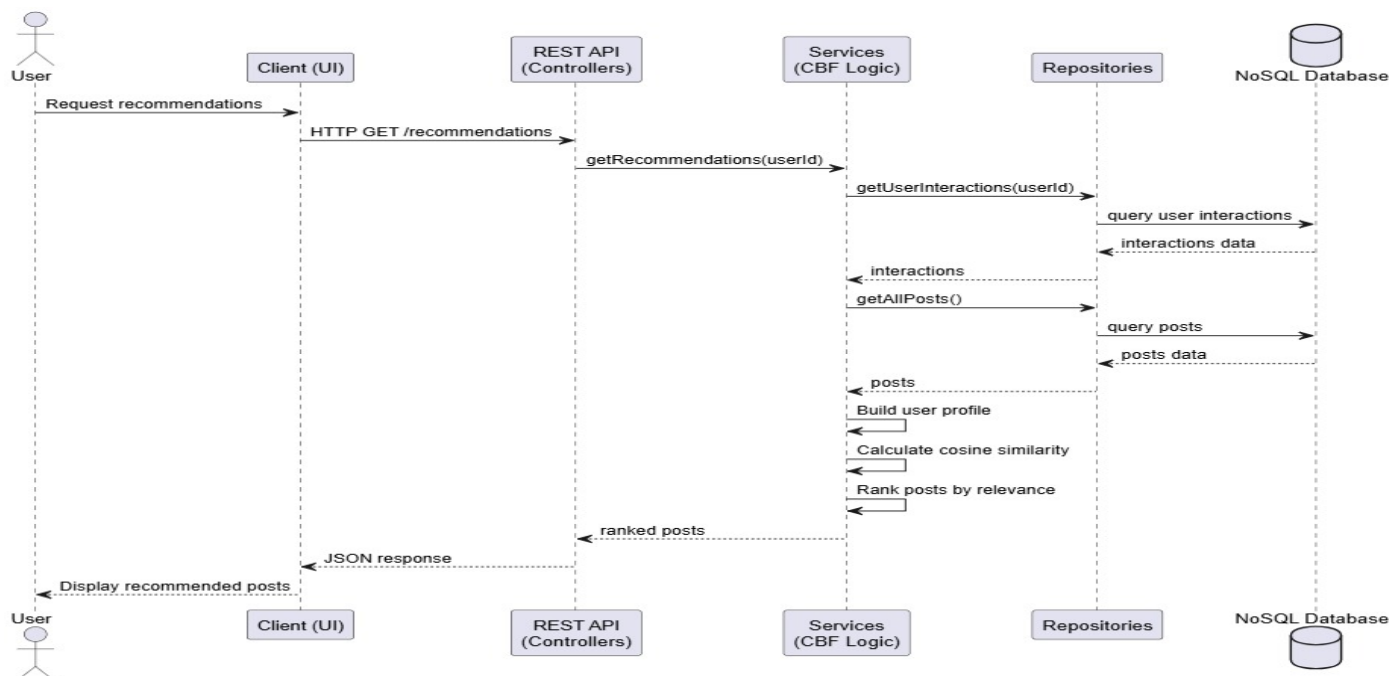
תיאור התרשים:

התרשים מציג את שלושת רכיבי המערכת המרכזיים ואת הקשרים ביניהם:

- הלקוח מתקשר עם השרת באמצעות HTTP/HTTPS.
- השרת מתקשר עם מסד הנתונים לצורך שליפה ושמירה של נתונים.
- אלגוריתם ההמלצות פועל כחלק משכבת הלוגיקה העסקית בשרת.

הסבר טקסטואלי קצר:

תרשים זה מדגים הפרדה ברורה בין שכבות המערכת ואת זרימת המידע ביניהן. מבנה זה מאפשר שינוי או החלפה של רכיב אחד (לדוגמה, מסד הנתונים) מבלי להשפיע על שאר המערכת.

**הסבר טקסטואלי קצר:**

תרשים הרצף מדגים את סדר הפעולות והאינטראקציה בין רכיבי המערכת בזמן הפעלת האלגוריתם. התרשים מבליט את תפקיד השרת כגורם המרכזי בעיבוד הלוגיקה והפעלת מנגנון ההמלצות.

6. הילכים עיקריים בתחום למידת מכונה (כולל ניתוח השוואתי) לא רלוונטי לפרויקט

לא רלוונטי לפרויקט.

7. הילכים עיקריים בתחום מערכות הפעלה/רשתות/אבטחה לא רלוונטי לפרויקט

לא רלוונטי לפרויקט.

8. פרטוקולי תקשורת

בפרויקט זה נעשה שימוש במספר פרטוקולי תקשורת סטנדרטיים, המאפשרים העברת מידע אמינה, מאובטחת ויעילה בין רכיבי המערכת השונים. הפרטוקולים נבחרו בהתאם לאופי המערכת, המבוססת על ארכיטקטורת Client-Server.

HTTP / HTTPS

פרוטוקול HTTP משמש כבסיס לתקשורת בין צד הלקוח (Front-End) לבין צד השרת (Back-End). התקשורת מתבצעת באמצעות בקשות (Requests) ותגובות (Responses) במבנה מוגדר.

במערכת זו נעשה שימוש ב־ HTTPS, המוסיף שכבת אבטחה באמצעות הצפנה (SSL/TLS), לצורך הגנה על נתוני המשתמשים והמערכת.

שימושים עיקריים:

- שליחת בקשות מהלקוח לשרת
- קבלת פוסטים, נתוני משתמשים ותוצאות אלגוריתם ההמלצות
- ביצוע פעולות משתמש כגון לייקים, צפיות וסימון מועדפים

REST API

המערכת מממשת ממשק REST API לצורך תקשורת בין הלקוח לשרת. ה־ API בנוי לפי עקרונות REST, כולל שימוש ב־ HTTP Methods והפרדה בין משאבים.

דוגמאות ל־ Endpoints מרכזיים במערכת:

- GET /posts – קבלת רשימת פוסטים
- GET /posts/recommended – קבלת פוסטים מומלצים למשתמש
- POST /interactions – שמירת אינטראקציית משתמש (צפייה, לייק)
- GET /users/{id} – שליפת נתוני משתמש
- POST /auth/login – התחברות למערכת

ה־ REST API אחראי לקבלת הבקשות, העברתן לשכבת הלוגיקה העסקית והחזרת תוצאות בפורמט אחיד.

פורמט התקשורת JSON –

הנתונים במערכת מועברים בין הלקוח לשרת בפורמט JSON (JavaScript Object Notation).

יתרונות השימוש ב־ JSON:

- פורמט קל לקריאה ולעיבוד
- התאמה טבעית למבני נתונים היררכיים
- תאימות גבוהה למסדי נתונים מסוג NoSQL
- תקשורת יעילה בין שכבות המערכת

כל בקשה ותגובה מכילות אובייקטי JSON המתארים פוסטים, משתמשים, תגיות ונתוני אינטראקציה.

TCP/IP

פרוטוקול TCP/IP מהווה את שכבת התשתית של כל התקשורת ברשת. פרוטוקול TCP אחראי להעברת מידע אמינה, מסודרת וללא אובדן נתונים, בעוד ש-IP אחראי לניתוב החבילות בין רכיבי המערכת.

בפרויקט זה:

- HTTP/HTTPS פועלים מעל TCP/IP
- התקשורת בין הלקוח לשרת ובין השרת למסד הנתונים מתבצעת מעל שכבה זו
- הפרוטוקול מבטיח יציבות ואמינות בהעברת הנתונים

סיכום

שילוב פרוטוקולי HTTP/HTTPS, REST API, JSON ו-TCP/IP מאפשר למערכת לפעול בצורה מאובטחת, מודולרית ויעילה. מבנה זה תומך בהפרדת שכבות, תחזוקה קלה והרחבה עתידית של המערכת.

9. פיתוחים עתידיים

רכיבים שלא נכללו בגרסה הנוכחית

- אלגוריתמי המלצה מתקדמים מבוססי למידת מכונה או ניתוח התנהגות משתמשים רחב היקף
- ניתוח אוטומטי של תוכן חזותי (כגון זיהוי צבעים, סגנון או אובייקטים בתמונה)
- מנגנון חיפוש מתקדם המבוסס על שקלול משוקלל של פרמטרים רבים
- מערכת התראות בזמן אמת (Real-Time Notifications)
- מנגנוני ניהול מתקדמים למנהלי מערכת (סטטיסטיקות, ניתוח שימוש)

הרחבות ופיתוחים אפשריים לעתיד

- **יישום אלגוריתם Collaborative Filtering**
שילוב ניתוח דפוסי שימוש של משתמשים שונים לצורך שיפור איכות ההמלצות, במיוחד במערכות עם בסיס משתמשים רחב.
- **מערכת היברידית להמלצות**
שילוב בין Content-Based Filtering לבין Collaborative Filtering לקבלת דיוק וגיוון גבוהים יותר בהמלצות.

- **ניתוח מאפיינים חזותיים של תמונות**
הוספת חילוץ מאפיינים חזותיים בסיסיים (כגון צבעוניות, בהירות או סגנון כללי) לצורך העשרת וקטור התכונות של הפוסטים.
- **ממשק משתמש מתקדם**
הוספת אפשרויות סינון, מיון והתאמה אישית מתקדמות לשיפור חוויית המשתמש.
- **תקשורת בזמן אמת**
שימוש בפרוטוקול WebSocket לצורך הצגת פוסטים חדשים, התראות ואינטראקציות בזמן אמת.
- **שיפור ביצועים וסקיילביליות**
אופטימיזציה של תהליכי שליפה, שימוש במנגנוני Cache והרחבת תשתיות המערכת לתמיכה בעומסים גבוהים.

10. תיאור טכנולוגיית הנדסה

שפת תכנות Java 21 –

Java 21 היא שפת תכנות עילית מונחית עצמים, (Object-Oriented Programming), המאפשרת פיתוח מערכות מורכבות תוך שמירה על עקרונות של תחזוקה, הרחבה ואבטחה. השפה מספקת ביצועים גבוהים, ניהול זיכרון מתקדם ותמיכה רחבה בפיתוח מערכות צד שרת, ולכן נבחרה כמנוע המרכזי של המערכת.

מסגרת עבודה Spring Boot 3.5.9 – (Framework)

Spring Boot היא מסגרת עבודה לפיתוח יישומי צד שרת מבוססי Java, המאפשרת פיתוח מהיר באמצעות תצורה אוטומטית, ניהול תלויות מובנה וארכיטקטורה ברורה. הבחירה ב-Spring Boot מאפשרת:

- מימוש REST API בצורה יעילה
- הפרדת שכבות (Controllers, Services, Repositories)
- אינטגרציה פשוטה עם מסדי נתונים
- הרחבה ותחזוקה עתידית של המערכת

סביבת פיתוח Visual Studio Code 1.108.0 – (IDE)

Visual Studio Code היא סביבת פיתוח קלה, גמישה ונוחה, התומכת בפיתוח Java באמצעות הרחבות ייעודיות. הסביבה מאפשרת:

- כתיבת קוד וניווט מהיר בפרויקט
- ניפוי שגיאות (Debugging)
- ניהול גרסאות באמצעות Git

- עבודה נוחה עם Spring Boot ו-MongoDB

ממשק משתמש Vaadin –

Vaadin היא מסגרת עבודה לפיתוח ממשקי Web אינטראקטיביים באמצעות Java בלבד. המסגרת מאפשרת בניית ממשק משתמש בצד השרת עם אינטגרציה מלאה ל-Spring Boot, ניהול מאובטח של הלוגיקה והנתונים. בחירת Vaadin מאפשרת:

- פיתוח Front-End ללא שימוש ב-JavaScript
- שמירה על לוגיקה עסקית בצד השרת
- אבטחה מובנית וניהול הרשאות

מסד נתונים MongoDB –

MongoDB הוא מסד נתונים מסוג NoSQL המבוסס על מסמכים (Document-Oriented Database).

המסד מאפשר אחסון מידע גמיש וסקלאבילי, ומתאים במיוחד ליישומים בעלי מבנה נתונים דינמי כגון פוסטים, תגיות ואינטראקציות משתמש.

MongoDB משתלב באופן טבעי עם Spring Boot ו-Spring Data.

הרחבות וכלי עזר (Extensions)

- **PlantUML** – כלי ליצירת תרשימים (תרשימי רצף, תרשימי ארכיטקטורה ו-ERD) באמצעות תיאור טקסטואלי.
- **VS Code** – הרחבות לפיתוח Java, Spring ו-MongoDB המסייעות לייעול תהליך הפיתוח, הבדיקה והתחזוקה.

פריית עיקריות (Dependencies)

- **Spring Web** – בניית REST API וטיפול בבקשות ותגובות HTTP
- **Vaadin** – בניית ממשק משתמש Web אינטראקטיבי
- **Spring Data MongoDB** – חיבור למסד הנתונים MongoDB וביצוע פעולות שמירה ושליפה
- **Spring Boot DevTools** – טעינה מחדש וכלי עזר לפיתוח מהיר

11. מסד נתונים

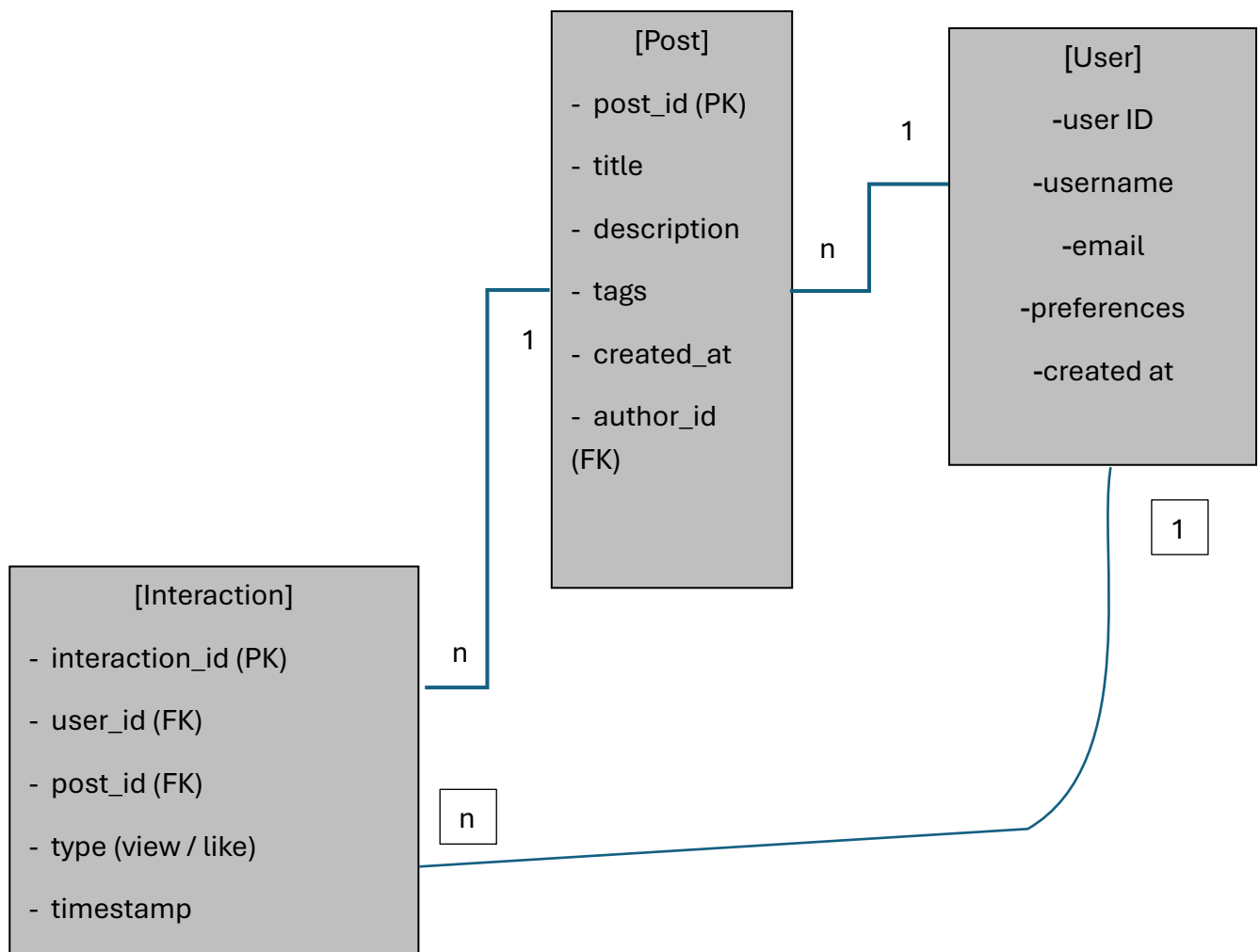
במערכת נעשה שימוש במסד נתונים לצורך אחסון נתוני משתמשים, פוסטים ואינטראקציות, המשמשים גם כבסיס לפעולת אלגוריתם ההמלצות.

סוג מסד הנתונים (MongoDB) NoSQL –

בפרויקט זה נעשה שימוש במסד נתונים מסוג NoSQL, ובפרט, MongoDB המבוסס על אחסון נתונים בפורמט מסמכים (Documents).

הבחירה ב־ MongoDB נובעת מהסיבות הבאות:

- גמישות גבוהה במבנה הנתונים
- התאמה לנתונים דינמיים כגון פוסטים, תגיות ואינטראקציות
- ביצועים טובים בשליפה של כמויות נתונים גדולות
- אינטגרציה טבעית עם Spring Boot ו־ Spring Data
- התאמה לאלגוריתם Content-Based Filtering הדורש הרחבה עתידית של מאפיינים



טבלאות / אוספים עיקריים (Collections)**User**

מכיל מידע על משתמשי המערכת.

שדות עיקריים:

- id – מזהה משתמש
- username – שם משתמש
- email – כתובת דוא"ל
- preferences – רשימת תחומי עניין / תגיות
- createdAt – תאריך יצירה

Post

מכיל מידע על פוסטים ותוכן חזותי.

שדות עיקריים:

- id – מזהה פוסט
- title – כותרת
- description – תיאור
- tags – תגיות תוכן
- authorId – מזהה יוצר הפוסט
- createdAt – תאריך פרסום

Interaction

מתעד אינטראקציות של משתמשים עם פוסטים, ומשמש כקלט לאלגוריתם ההמלצות.

שדות עיקריים:

- id – מזהה אינטראקציה
- userId – מזהה משתמש
- postId – מזהה פוסט
- type – אינטראקציה (צפייה, לייק)
- timestamp – ביצוע

הסבר הקשרים בין הישגיות

- משתמש אחד יכול ליצור מספר פוסטים
- משתמש אחד יכול לבצע מספר אינטראקציות
- כל אינטראקציה מקשרת בין משתמש לפוסט
- הנתונים מהאינטראקציות משמשים לבניית פרופיל משתמש לצורך חישוב המלצות

12. פרטים פורמליים

לוח זמנים:

לסיים עד תאריך	שלבי עבודה
1.12.2025	בחירת פרויקט, חקירה ולמידה לעומק של נושאי הפרויקט
11.12.2025	כתיבה והגשת הצעת הפרויקט לאישור משרד החינוך
13.1.2026	מימוש הקוד של האלגוריתם המרכזי, ביצוע בדיקות ושיפורים
3.2.2026	בניית צד שרת
17.2.2026	בניית מסד הנתונים ושילובו
3.3.2026	בניית צד לקוח
24.3.2026	כתיבת ספר הפרויקט
7.4.2026	הגשת הפרויקט כולו (ספר + קוד) להגנה וקבלת ציון מגן

מנחה הפרויקט: אילן פרץ



חתימת הסטודנט:

חתימת רכז המגמה: